Team Number: 25

VANGUARD: COMMUNITY STEWARDSHIP

M1 — Approach

1. Problem Statement and Understanding

Students need practical financial-literacy education that connects financial concepts to everyday decisions. The challenge is to design an accessible, engaging digital experience for students in Grades 11-12, particularly from underserved communities across Hyderabad, that builds understanding of budgeting, saving, basic investing concepts, and financial safety. The platform should make learning simple and relatable, enable students to practice decisions without real-world financial consequences, and provide a way to evaluate what they have learned. It must remain educational rather than recommend investments, protect student privacy, and be usable with minimal guidance.

2. Idea / Abstract

RupeeRise is proposed as a student-centered financial-literacy web application that combines structured learning with an interactive, branching story. Students follow relatable school and community characters facing limited budgets, competing needs, unexpected expenses, online offers, and shared-money decisions. At key points, students choose how a character should act, observe the consequences, and receive short explanations. The story is designed to make financial knowledge and ethical reasoning visible through decisions rather than passive viewing alone.

The experience will cover four learning pillars: (1) Plan and Budget, including needs versus wants and goals; (2) Save and Manage, including habits and unexpected expenses; (3) Understand Investing, introducing risk, time, compounding, and diversification at an age-appropriate level; and (4) Stay Financially Safe, including protecting personal information and recognising suspicious offers. A short pre-assessment and a post-assessment using different questions will estimate learning progress. Financial knowledge and ethical reasoning will be reported separately, with supportive feedback and suggested areas for practice.

3. High-Level Approach and Architecture

The solution will be developed as a responsive web application. The frontend will use React and TypeScript with Vite, providing the dashboard, learning areas, story scenes, decision controls, assessments, and progress views. A backend API built with Node.js and Express will validate requests, apply learning and scoring rules, and coordinate data access. PostgreSQL through Supabase is the proposed persistent data layer for learning content, activity completion, and assessment results. These technologies are the planned implementation stack and may be adjusted to fit the final build.

Planned pipeline

- **1. Student entry:** present an accessible dashboard and age-appropriate learning pathway.
- **2. Baseline:** run a short diagnostic assessment to estimate existing understanding.
- **3. Learn by doing:** deliver branching story scenes with decisions, simulated outcomes, and explanatory feedback.
- **4. Measure:** assess the same learning objectives with new questions and evaluate financial knowledge separately from ethical reasoning.
- **5. Persist and review:** save permitted results and show topic-level progress and recommended practice.

3.1 System Components and Data Flow

The browser-based frontend will communicate with the backend over HTTP using JSON request and response formats. The backend will expose endpoints for retrieving learning areas and lessons, submitting assessment results, running budget simulations, and retrieving progress. The backend will validate inputs and calculate authoritative results before saving them. The database will use related records for learning areas, lessons, activities, student identifiers where required, completion status, and assessment results. During early UI development, mock data will allow the frontend to progress independently; it will be replaced by tested API calls during integration. No real bank accounts or financial transactions are part of the proposed system.

4. Planned Testing Methodology

Testing will be conducted throughout development, first on individual components and then across the complete student journey. The planned methodology includes:

- **Functional testing:** verify navigation, story branching, answer submission, budget arithmetic, feedback, and progress updates.
- **Assessment and scoring checks:** use known test cases to confirm answer keys, scoring rules, separate financial and ethical dimensions, and pre/post comparison calculations.
- **API testing:** test each endpoint with valid, missing, malformed, and out-of-range inputs; verify status codes, response structures, and error handling.
- **Database testing:** confirm that permitted progress and results are saved and retrieved correctly, relationships are valid, and duplicate records are handled as intended.
- **Integration testing:** complete an end-to-end flow from opening a learning area through making story decisions, submitting a post-assessment, and viewing saved progress after refresh.
- **Usability and accessibility testing:** check mobile and desktop layouts, readable language, keyboard navigation, subtitles or text alternatives, and clear feedback for students in Grades 7–11.
- **Privacy and safety checks:** use fictional data, minimise personal information, keep secrets on the server, verify access controls where accounts are used, and ensure the experience does not recommend investments or promise returns.

Test results will be recorded as pass/fail findings with defects prioritised by impact. The final demonstration will use a repeatable test account or fictional dataset and will be run against the deployed website, not only the local development version.

5. Expected Outcome

The intended outcome is a working, responsive prototype that combines interactive financial learning, safe simulated decisions, age-appropriate explanations, and a basic measurement of learning progress. Success will be judged by whether the core student journey functions end to end, the assessment and calculations behave correctly, progress persists where implemented, and the interface remains understandable and privacy-conscious. Advanced personalization, additional languages, and NGO-facing reporting can be expanded after the core experience is stable.